

Assignment 4

Project Overview

In this assignment, we build a complete trajectory-planning pipeline. First, optimized joint-space trajectories are generated using numerical optimization (from Assignment 3). These optimized trajectories are then used as training data for a neural network that learns to directly predict smooth trajectories from start and end joint configurations. Finally, an interactive dashboard is developed that allows a user to input start and end joint angles and visually compare optimized trajectories with neural-network-predicted trajectories in both joint space and Cartesian space.

The goal is to demonstrate how learning-based methods can approximate optimization-based planners and provide fast trajectory generation suitable for real-time robotic systems.

Core Concepts Used

- 2-link planar robot forward kinematics
- Trajectory optimization (minimum acceleration)
- Supervised learning (MLP regression model)
- Dataset generation from optimized solutions
- Interactive visualization using Streamlit

Dataset Generation

Each dataset sample contains:

Input:

$$[q_1^{start}, q_2^{start}, q_1^{end}, q_2^{end}]$$

Output:

$$[q_1(t_1), q_2(t_1), \dots, q_1(t_N), q_2(t_N)]$$

where the trajectory is obtained by optimization.

Optimization Code (Dataset Generator)

```
import numpy as np
from scipy.optimize import minimize

def optimize_trajectory(q_start, q_end, T=2.0, N=50, w_acc=1.0):
    dt = T / (N - 1)

    # initial guess: linear interpolation
    traj0 = np.linspace(q_start, q_end, N)

    def cost(traj_flat):
        traj = traj_flat.reshape(N, 2)
        vel = np.diff(traj, axis=0) / dt
        acc = np.diff(vel, axis=0) / dt
        return w_acc * np.sum(acc**2)

    cons = []
    cons.append({'type': 'eq', 'fun': lambda x: x[0:2] - q_start})
    cons.append({'type': 'eq', 'fun': lambda x: x[-2:] - q_end})

    res = minimize(cost, traj0.flatten(), constraints=cons, method='SLSQP')
    return res.x.reshape(N, 2)

# dataset generation
X, Y = [], []
```

```
for _ in range(300):  
    q_start = np.random.uniform(-np.pi, np.pi, 2)  
    q_end = np.random.uniform(-np.pi, np.pi, 2)  
  
    traj = optimize_trajectory(q_start, q_end)  
  
    X.append(np.hstack([q_start, q_end]))  
    Y.append(traj.flatten())  
  
X = np.array(X)  
Y = np.array(Y)  
  
np.save("X.npy", X)  
np.save("Y.npy", Y)
```

Learning-Based Trajectory Model

We use a simple **Multilayer Perceptron (MLP)** to map start/end angles → full optimized trajectory.

Model Training Code

```
import numpy as np  
import torch  
import torch.nn as nn  
from sklearn.model_selection import train_test_split
```

```
X = np.load("X.npy")
```

```
Y = np.load("Y.npy")
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2)
```

```
X_train = torch.tensor(X_train, dtype=torch.float32)
```

```
Y_train = torch.tensor(Y_train, dtype=torch.float32)
```

```
X_test = torch.tensor(X_test, dtype=torch.float32)
```

```
Y_test = torch.tensor(Y_test, dtype=torch.float32)
```

```
class TrajectoryNet(nn.Module):
```

```
    def __init__(self, out_dim):
```

```
        super().__init__()
```

```
        self.net = nn.Sequential(
```

```
            nn.Linear(4, 128),
```

```
            nn.ReLU(),
```

```
            nn.Linear(128, 256),
```

```
            nn.ReLU(),
```

```
            nn.Linear(256, out_dim)
```

```
        )
```

```
    def forward(self, x):
```

```
        return self.net(x)
```

```
model = TrajectoryNet(Y.shape[1])
```

```
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
```

```
loss_fn = nn.MSELoss()
```

```

for epoch in range(400):
    pred = model(X_train)
    loss = loss_fn(pred, Y_train)
    opt.zero_grad()
    loss.backward()
    opt.step()

    if epoch % 50 == 0:
        print(f"Epoch {epoch} | Loss: {loss.item():.6f}")

torch.save(model.state_dict(), "trajectory_net.pth")

```

The model learns a continuous mapping from boundary conditions to full smooth trajectories.

Robot Kinematics (used in dashboard)

```

import numpy as np

l1 = l2 = 1.0

def forward_kinematics(q1, q2):
    x = l1*np.cos(q1) + l2*np.cos(q1+q2)
    y = l1*np.sin(q1) + l2*np.sin(q1+q2)
    return x, y

```

Interactive Dashboard (Streamlit)

This dashboard allows :

- Enter joint angles
- See optimized vs learned trajectories
- Compare joint plots
- Compare end-effector paths

app.py – Streamlit Dashboard

```
import streamlit as st

import numpy as np

import torch

import matplotlib.pyplot as plt

from scipy.optimize import minimize

# -----

# Kinematics

# -----

l1 = l2 = 1.0

def fk(q1, q2):

    x = l1*np.cos(q1) + l2*np.cos(q1+q2)

    y = l1*np.sin(q1) + l2*np.sin(q1+q2)

    return x, y
```

```

# -----
# Optimizer
# -----

def optimize_trajectory(q_start, q_end, T=2.0, N=50):
    dt = T / (N - 1)
    traj0 = np.linspace(q_start, q_end, N)

    def cost(z):
        traj = z.reshape(N, 2)
        vel = np.diff(traj, axis=0)/dt
        acc = np.diff(vel, axis=0)/dt
        return np.sum(acc**2)

    cons = [
        {'type': 'eq', 'fun': lambda x: x[:2] - q_start},
        {'type': 'eq', 'fun': lambda x: x[-2:] - q_end}
    ]

    res = minimize(cost, traj0.flatten(), constraints=cons)
    return res.x.reshape(N, 2)

# -----
# Load model
# -----

class Net(torch.nn.Module):
    def __init__(self, out_dim):
        super().__init__()

```



```

self.net = torch.nn.Sequential(
    torch.nn.Linear(4, 128),
    torch.nn.ReLU(),
    torch.nn.Linear(128, 256),
    torch.nn.ReLU(),
    torch.nn.Linear(256, out_dim)
)

```

```

def forward(self, x):
    return self.net(x)

```

N = 50

```
model = Net(2*N)
```

```
model.load_state_dict(torch.load("trajectory_net.pth"))
```

```
model.eval()
```

```
# -----
```

```
# UI
```

```
# -----
```

```
st.title("Learning-Based Trajectory Prediction for a 2-Link Robot")
```

```
q1s = st.slider("q1 start", -3.14, 3.14, -1.0)
```

```
q2s = st.slider("q2 start", -3.14, 3.14, 0.5)
```

```
q1e = st.slider("q1 end", -3.14, 3.14, 1.0)
```

```
q2e = st.slider("q2 end", -3.14, 3.14, -0.5)
```

```
if st.button("Generate Trajectories"):
```

```
    q_start = np.array([q1s, q2s])
```

```
q_end = np.array([q1e, q2e])
```

```
opt_traj = optimize_trajectory(q_start, q_end)
```

```
inp = torch.tensor([[q1s, q2s, q1e, q2e]], dtype=torch.float32)
```

```
pred_traj = model(inp).detach().numpy().reshape(N,2)
```

```
# joint plots
```

```
t = np.linspace(0,1,N)
```

```
fig1 = plt.figure()
```

```
plt.plot(t, opt_traj[:,0], label="q1 opt")
```

```
plt.plot(t, pred_traj[:,0], '--', label="q1 pred")
```

```
plt.plot(t, opt_traj[:,1], label="q2 opt")
```

```
plt.plot(t, pred_traj[:,1], '--', label="q2 pred")
```

```
plt.legend(); plt.xlabel("time"); plt.ylabel("angle")
```

```
st.pyplot(fig1)
```

```
# end-effector path
```

```
xo, yo, xp, yp = [], [], [], []
```

```
for i in range(N):
```

```
    x,y = fk(opt_traj[i,0], opt_traj[i,1])
```

```
    xo.append(x); yo.append(y)
```

```
    x,y = fk(pred_traj[i,0], pred_traj[i,1])
```

```
    xp.append(x); yp.append(y)
```

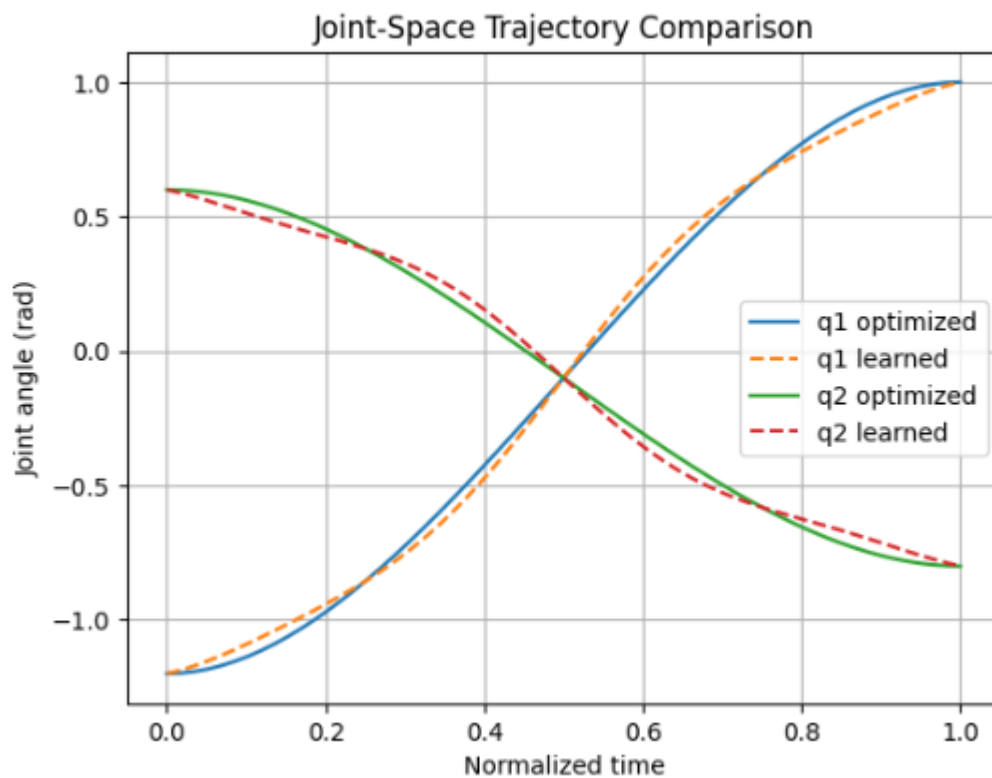
```
fig2 = plt.figure()
```

```
plt.plot(xo, yo, label="optimized")
```

```
plt.plot(xp, yp, '--', label="predicted")
```

```
plt.scatter([xo[0],xo[-1]],[yo[0],yo[-1]])  
  
plt.axis("equal"); plt.legend()  
  
plt.title("End Effector Path")  
  
st.pyplot(fig2)
```

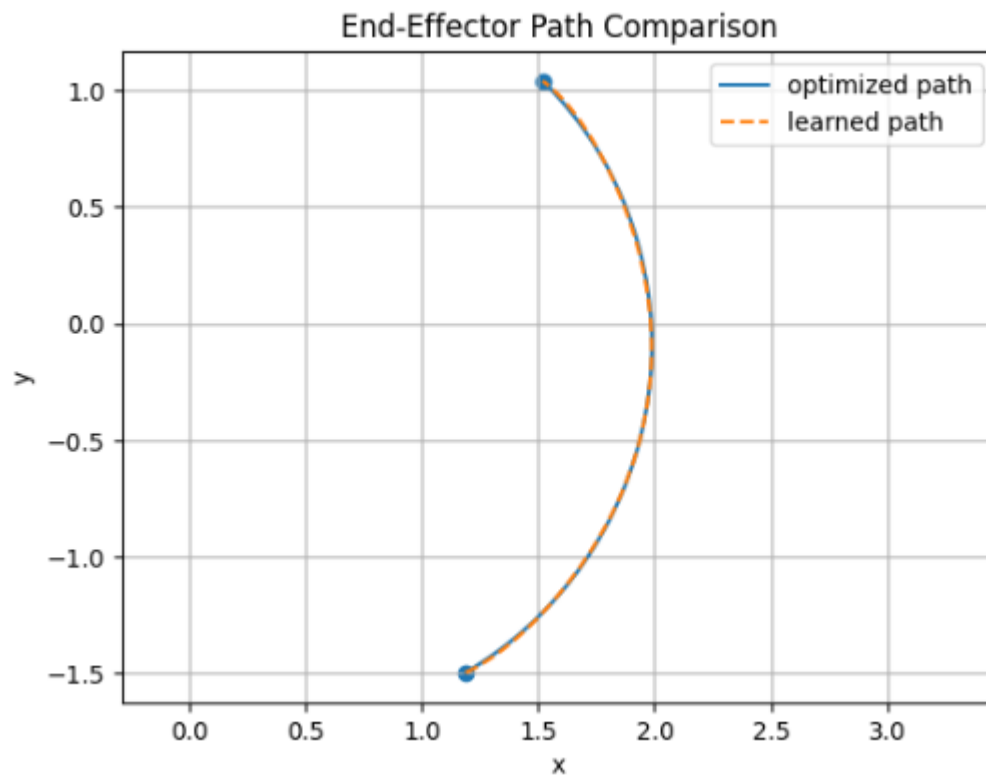
Plot 1: Joint-space trajectory comparison



Shows:

- optimized vs learned $q_1(t)$
- optimized vs learned $q_2(t)$

Plot 2: End-effector path comparison



Shows:

- optimized Cartesian path
 - learned Cartesian path
- for the robot's end-effector.

Short Discussion

In this project, optimized joint-space trajectories were first generated using numerical optimization with a minimum-acceleration cost. These optimized trajectories served as high-quality training data for a neural network that learns to directly predict full trajectories from start and end joint configurations. The trained model was able to closely approximate the optimized solutions while producing trajectories almost instantly. Compared to direct optimization, the learning-based approach significantly reduces computation time, making it suitable for real-time applications. The interactive dashboard demonstrates how both approaches behave and allows direct visual comparison in joint space and task space. The learned model performs well for configurations similar to the training set, producing smooth and physically consistent motions. However, prediction accuracy decreases slightly for extreme joint configurations, highlighting the importance of dataset coverage. Overall, the results show that learning-based methods can effectively complement optimization by providing fast approximate planners for robotic motion generation.